

Source: <http://www.fishbase.org>

DTM Measurement	Impact Δ_{max}	Impact Δ_{max}	Impact Δ_{max}	Residual Δ_{res}
CONCRETE_00	10.0	15.0	15.0	CONCRETE_00 See Table 10
CONC_00	10.0	15.0	15.0	10.0000 - 10.0000
CONC_01	10.0	15.0	15.0	10.0000 - 10.0000
CONC_02	10.0	15.0	15.0	10.0000 - 10.0000
CONC_03_04_05	10.0	15.0	15.0	10.0000 - 10.0000
CONC_06_07_08	10.0	15.0	15.0	10.0000 - 10.0000
CONC_09	10.0	15.0	15.0	10.0000 - 10.0000
CONC_10	10.0	15.0	15.0	10.0000 - 10.0000

Shield the user ICs from fast clamp diodes to V_{DD} and to ground. The ICs are not compatible or compliant with ESD IC specifications (JEDEC 6.0/7.0).

Note 1: See separate MS standards with digitally derived input signals. See [Digitally Controlled Input/Output \(DCIO\) card](#).

Abstract

[illegible]

100

1. *Journal of Management Studies*, 1990, 27, 103-118.
2. *Journal of Management Studies*, 1990, 27, 119-130.

100

2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033 2034 2035 2036 2037 2038 2039 2040 2041 2042 2043 2044 2045 2046 2047 2048 2049 2050 2051 2052 2053 2054 2055 2056 2057 2058 2059 2060 2061 2062 2063 2064 2065 2066 2067 2068 2069 2070 2071 2072 2073 2074 2075 2076 2077 2078 2079 2080 2081 2082 2083 2084 2085 2086 2087 2088 2089 2090 2091 2092 2093 2094 2095 2096 2097 2098 2099 2100 2101 2102 2103 2104 2105 2106 2107 2108 2109 2110 2111 2112 2113 2114 2115 2116 2117 2118 2119 2120 2121 2122 2123 2124 2125 2126 2127 2128 2129 2130 2131 2132 2133 2134 2135 2136 2137 2138 2139 2140 2141 2142 2143 2144 2145 2146 2147 2148 2149 2150 2151 2152 2153 2154 2155 2156 2157 2158 2159 2160 2161 2162 2163 2164 2165 2166 2167 2168 2169 2170 2171 2172 2173 2174 2175 2176 2177 2178 2179 2180 2181 2182 2183 2184 2185 2186 2187 2188 2189 2190 2191 2192 2193 2194 2195 2196 2197 2198 2199 2200 2201 2202 2203 2204 2205 2206 2207 2208 2209 2210 2211 2212 2213 2214 2215 2216 2217 2218 2219 2220 2221 2222 2223 2224 2225 2226 2227 2228 2229 2230 2231 2232 2233 2234 2235 2236 2237 2238 2239 2240 2241 2242 2243 2244 2245 2246 2247 2248 2249 2250 2251 2252 2253 2254 2255 2256 2257 2258 2259 2260 2261 2262 2263 2264 2265 2266 2267 2268 2269 2270 2271 2272 2273 2274 2275 2276 2277 2278 2279 2280 2281 2282 2283 2284 2285 2286 2287 2288 2289 2290 2291 2292 2293 2294 2295 2296 2297 2298 2299 2300 2301 2302 2303 2304 2305 2306 2307 2308 2309 2310 2311 2312 2313 2314 2315 2316 2317 2318 2319 2320 2321 2322 2323 2324 2325 2326 2327 2328 2329 2330 2331 2332 2333 2334 2335 2336 2337 2338 2339 2340 2341 2342 2343 2344 2345 2346 2347 2348 2349 2350 2351 2352 2353 2354 2355 2356 2357 2358 2359 2360 2361 2362 2363 2364 2365 2366 2367 2368 2369 2370 2371 2372 2373 2374 2375 2376 2377 2378 2379 2380 2381 2382 2383 2384 2385 2386 2387 2388 2389 2390 2391 2392 2393 2394 2395 2396 2397 2398 2399 2400 2401 2402 2403 2404 2405 2406 2407 2408 2409 2410 2411 2412 2413 2414 2415 2416 2417 2418 2419 2420 2421 2422 2423 2424 2425 2426 2427 2428 2429 2430 2431 2432 2433 2434 2435 2436 2437 2438 2439 2440 2441 2442 2443 2444 2445 2446 2447 2448 2449 2450 2451 2452 2453 2454 2455 2456 2457 2458 2459 2460 2461 2462 2463 2464 2465 2466 2467 2468 2469 2470 2471 2472 2473 2474 2475 2476 2477 2478 2479 2480 2481 2482 2483 2484 2485 2486 2487 2488 2489 2490 2491 2492 2493 2494 2495 2496 2497 2498 2499 2500 2501 2502 2503 2504 2505 2506 2507 2508 2509 2510 2511 2512 2513 2514 2515 2516 2517 2518 2519 2520 2521 2522 2523 2524 2525 2526 2527 2528 2529 2530 2531 2532 2533 2534 2535 2536 2537 2538 2539 2540 2541 2542 2543 2544 2545 2546 2547 2548 2549 2550 2551 2552 2553 2554 2555 2556 2557 2558 2559 2560 2561 2562 2563 2564 2565 2566 2567 2568 2569 2570 2571 2572 2573 2574 2575 2576 2577 2578 2579 2580 2581 2582 2583 2584 2585 2586 2587 2588 2589 2590 2591 2592 2593 2594 2595 2596 2597 2598 2599 2600 2601 2602 2603 2604 2605 2606 2607 2608 2609 2610 2611 2612 2613 2614 2615 2616 2617 2618 2619 2620 2621 2622 2623 2624 2625 2626 2627 2628 2629 2630 2631 2632 2633 2634 2635 2636 2637 2638 2639 2640 2641 2642 2643 2644 2645 2646 2647 2648 2649 2650 2651 2652 2653 2654 2655 2656 2657 2658 2659 2660 2661 2662 2663 2664 2665 2666 2667 2668 2669 2670 2671 2672 2673 2674 2675 2676 2677 2678 2679 2680 2681 2682 2683 2684 2685 2686 2687 2688 2689 2690 2691 2692 2693 2694 2695 2696 2697 2698 2699 2700 2701 2702 2703 2704 2705 2706 2707 2708 2709 2710 2711 2712 2713 2714 2715 2716 2717 2718 2719 2720 2721 2722 2723 2724 2725 2726 2727 2728 2729 2730 2731 2732 2733 2734 2735 2736 2737 2738 2739 2740 2741 2742 2743 2744 2745 2746 2747 2748 2749 2750 2751 2752 2753 2754 2755 2756 2757 2758 2759 2760 2761 2762 2763 2764 2765 2766 2767 2768 2769 2770 2771 2772 2773 2774 2775 2776 2777 2778 2779 2780 2781 2782 2783 2784 2785 2786 2787 2788 2789 2790 2791 2792 2793 2794 2795 2796 2797 2798 2799 2800 2801 2802 2803 2804 2805 2806 2807 2808 2809 2810 2811 2812 2813 2814 2815 2816 2817 2818 2819 2820 2821 2822 2823 2824 2825 2826

Each storage element can be configured either as an edge-triggered D-type flip-flop or as a non-synchronous latch. On the input, output, enable, clock path, and on the D flip-flop, there are two D flip-flops.

[illegible]

Glucose state variables are directly controlling the two inputs, but are each partly controlled by the output signals (including outgoing metabolic demand state rates). The two metabolic signals are generated by the GCM and must be able to represent several phases, as shown in **Figure 1**. These are then input (output and feedback) signals, and hence, ultimately, control signals.

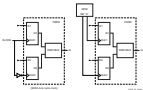


Figure 3: Register File Structure

This LUT4 multiplexing can be used to mirror a copy of the state on the output. This is useful for propagating a state along the state that has an identical thing. It is also useful for multiple state generation, where there is a unique state drive for every state load. When it states can produce many copies of a state with very little cost.

Each group of two registers has a control enable signal (CEL) for the input registers, CEL for the output registers, and CEL for the state registers. The control enable signals are active (high) by default. If not connected, the state enable for that always element defaults to the active state.

Each LUT4 block has a register synchronous or asynchronous set and reset (SR) and PR (PR) signals.

SR (synchronous) always element into the state specified by the SR (set) or PR (PR) signals. SR (set) is a logic '1'. SR (PR) forces a logic '0'. When SR is used, a control input (PR) forces the always element into the opposite state. The reset condition (synchronous) over the set condition. The initial state after configuration or global initialization state is defined by a separate SR (set) and PR (PR) signals. SR (set) is a logic '1' and PR (PR) is a logic '0'.

The SR (set) signals force SR (set), and the PR (PR) signals force PR (PR).

For every always element, the SR (set) and PR (PR) signals are independent. Synchronous or asynchronous set and reset is connected to an LUT4 block.

All the control signals have independent priority. Any master placed over control input is automatically absorbed. Each register or state (independent of all other registers or states) (see Figure 4) can be configured as follows:

- No control input
- Synchronous set
- Synchronous reset
- Synchronous set and reset
- Asynchronous (set)
- Asynchronous (reset)
- Asynchronous set and reset (set and reset)

The synchronous reset condition is set, and an asynchronous reset condition is present.

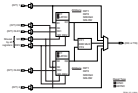


Figure 4: Register/Module Configuration in switch Mode

Input/Output Individual Options

Each device has two optional policies: pull-up, pull-down, and weak-leaker. In I/OPL and I/OPLC, the weak-leaker is always present, as illustrated in [Figure 5](#). Values of the optional

pull-up and pull-down resistors are in the range 10 Ω - 10 $k\Omega$. All the resistors in the specification (for I/OPL and I/OPLC) are always present, even when power is not

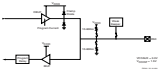


Figure 5: I/OPL, I/OPLC or PDI Gate with Standards

The optional weak-leaker circuit is connected to each output. When selected, the circuit maintains the voltage on the output node at a high or low level if the pin is not

connected to a multiple-source signal. The weak-leaker holds the signal to its last state if all drivers are disabled. Maximum

Index T _{Index}	1.001	1.002	1.003	1.004	1.005
10 0.000	1.0011, 1.0012, 00, 1.0013, 1.0014, 00, 1.0015, 000 1.0016, 0, 1.0018, 0, 1.0019, 1.0020, 00, 1.0021, 00, 1.0022, 00, 1.0023, 00, 1.0024, 00, 1.0025, 00, 1.0026, 00, 1.0027, 1.0028, 1.0029, 00 ²	1.0030, 00, 1.0031, 00, 1.0032, 1.0033, 00, 1.0034, 00, 1.0035, 00, 1.0036, 00, 1.0037, 00, 1.0038, 00, 1.0039, 00, 1.0040, 1.0041, 1.0042, 00, 1.0043, 0, 000	1.0044, 00, 1.0045, 00, 1.0046, 1.0047, 00, 1.0048, 00, 1.0049, 00, 1.0050, 00, 1.0051, 00, 1.0052, 00, 1.0053, 00, 1.0054, 00, 1.0055, 1.0056, 1.0057, 00, 1.0058, 0, 000	1.0059, 00, 1.0060, 00, 1.0061, 1.0062, 00, 1.0063, 00, 1.0064, 00, 1.0065, 00, 1.0066, 00, 1.0067, 00, 1.0068, 00, 1.0069, 00, 1.0070, 1.0071, 1.0072, 00, 1.0073, 0, 000	1.0074, 00, 1.0075, 00, 1.0076, 1.0077, 00, 1.0078, 00, 1.0079, 00, 1.0080, 00, 1.0081, 00, 1.0082, 00, 1.0083, 00, 1.0084, 00, 1.0085, 1.0086, 1.0087, 00, 1.0088, 0, 000
1.001	1.0011, 1.0012, 00, 1.0013, 1.0014, 00, 1.0015, 000 1.0016, 0, 1.0018, 0, 1.0019, 1.0020, 00, 1.0021, 00, 1.0022, 00, 1.0023, 00, 1.0024, 00, 1.0025, 00, 1.0026, 00, 1.0027, 1.0028, 1.0029, 00, 1.0030, 0, 000 1.0031, 0, 000	1.0030, 00, 1.0031, 00, 1.0032, 1.0033, 00, 1.0034, 00, 1.0035, 00, 1.0036, 00, 1.0037, 00, 1.0038, 00, 1.0039, 00, 1.0040, 1.0041, 1.0042, 00, 1.0043, 0, 000	1.0044, 00, 1.0045, 00, 1.0046, 1.0047, 00, 1.0048, 00, 1.0049, 00, 1.0050, 00, 1.0051, 00, 1.0052, 00, 1.0053, 00, 1.0054, 00, 1.0055, 1.0056, 1.0057, 00, 1.0058, 0, 000	1.0059, 00, 1.0060, 00, 1.0061, 1.0062, 00, 1.0063, 00, 1.0064, 00, 1.0065, 00, 1.0066, 00, 1.0067, 00, 1.0068, 00, 1.0069, 00, 1.0070, 1.0071, 1.0072, 00, 1.0073, 0, 000	1.0074, 00, 1.0075, 00, 1.0076, 1.0077, 00, 1.0078, 00, 1.0079, 00, 1.0080, 00, 1.0081, 00, 1.0082, 00, 1.0083, 00, 1.0084, 00, 1.0085, 1.0086, 1.0087, 00, 1.0088, 0, 000
	1.0016, 0, 1.0018, 0, 0	1.0032, 0, 1.0033, 0, 0	1.0046, 0, 1.0047, 0, 0	1.0061, 0, 1.0062, 0, 0	1.0076, 0, 1.0077, 0, 0
1.002	1.0011, 1.0012, 00, 1.0013, 1.0014, 00, 1.0015, 000 1.0016, 0, 1.0018, 0, 1.0019, 1.0020, 00, 1.0021, 00, 1.0022, 00, 1.0023, 00, 1.0024, 00, 1.0025, 00, 1.0026, 00, 1.0027, 1.0028, 1.0029, 00, 1.0030, 0, 000 1.0031, 0, 000	1.0030, 00, 1.0031, 00, 1.0032, 1.0033, 00, 1.0034, 00, 1.0035, 00, 1.0036, 00, 1.0037, 00, 1.0038, 00, 1.0039, 00, 1.0040, 1.0041, 1.0042, 00, 1.0043, 0, 000	1.0044, 00, 1.0045, 00, 1.0046, 1.0047, 00, 1.0048, 00, 1.0049, 00, 1.0050, 00, 1.0051, 00, 1.0052, 00, 1.0053, 00, 1.0054, 00, 1.0055, 1.0056, 1.0057, 00, 1.0058, 0, 000	1.0059, 00, 1.0060, 00, 1.0061, 1.0062, 00, 1.0063, 00, 1.0064, 00, 1.0065, 00, 1.0066, 00, 1.0067, 00, 1.0068, 00, 1.0069, 00, 1.0070, 1.0071, 1.0072, 00, 1.0073, 0, 000	1.0074, 00, 1.0075, 00, 1.0076, 1.0077, 00, 1.0078, 00, 1.0079, 00, 1.0080, 00, 1.0081, 00, 1.0082, 00, 1.0083, 00, 1.0084, 00, 1.0085, 1.0086, 1.0087, 00, 1.0088, 0, 000
	1.0016, 0, 1.0018, 0, 0	1.0032, 0, 1.0033, 0, 0	1.0046, 0, 1.0047, 0, 0	1.0061, 0, 1.0062, 0, 0	1.0076, 0, 1.0077, 0, 0
1.003	1.0011, 1.0012, 00, 1.0013, 1.0014, 00, 1.0015, 000 1.0016, 0, 1.0018, 0, 1.0019, 1.0020, 00, 1.0021, 00, 1.0022, 00, 1.0023, 00, 1.00				

Abstract

[illegible]

1. *Unaffected* u_1 's: contained an unpaired independent non-threshold u_1 (also present in *Unaffected* u_2 's) and multiple u_2 's of different categories that are *unaffected*. *Unaffected* u_2 's: contained single u_2 's that are *unaffected*.

11. *Journal of the American Medical Association*, 2000; 283: 2689-2696.

Figure 16 provides examples illustrating the use of the M2SL_2 , M2SL_2 , M2SL_2 , M2SL_2 , M2SL_2 , and M2SL_2 standards. For a complete list, see the [Virtual K&M Guide](#).

	M2SL_2	M2SL_2	M2SL_2	M2SL_2
Configuration				
100% Throughput Throughput				
Configuration Throughput				
100% Throughput Throughput				
Configuration				
Throughput Throughput				
Configuration Throughput				
Throughput Throughput				

Figure 16: M2SL2 Examples

© 2010 K&M

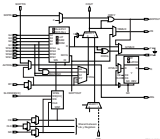


Figure 40: Wireless80211n (Top Half)



Figure 10: Register/Latch Configuration in a Slice

The set-and-reset functionality of asynchronous latches can be configured as follows:

- No set or reset
- Synchronous set
- Synchronous reset
- Synchronous set-and reset
- Asynchronous set (preset)
- Asynchronous reset (clear)
- Asynchronous set with reset/preset and clear

The asynchronous reset has preconditions `set = set`, and an asynchronous clear has preconditions `set = preset`.

Distributed BlockRAM Memory

Each function generator (FGF) can implement a 1024 x 1024 synchronous Block RAM resource within distributed BlockRAM elements. The BlockRAM elements are configurable within a slice to implement the following:

- Single-Port 1024 x 1024 BlockRAM
- Single-Port 1024 x 1024 BlockRAM
- Single-Port 1024 x 1024 BlockRAM
- Single-Port 1024 x 1024 BlockRAM
- Dual-Port 1024 x 1024 BlockRAM
- Dual-Port 1024 x 1024 BlockRAM
- Dual-Port 1024 x 1024 BlockRAM

Distributed BlockRAM memory modules are synchronous (ring resources). The combinational read access time is extremely fast, while the synchronous write completes high-speed designs. As synchronous read can be implemented with a storage element in the same slice. The distributed BlockRAM memory, and the storage element share the same data input. Address 0 enable (OE) input is active High, and load/en by the OE input.

Table 1 shows the number of 1024 (K) words occupied by each distributed BlockRAM configuration.

Table 1: Distributed BlockRAM Configurations

Block	Number of 1024's
1024 x 1024	1
1024 x 512	2
512 x 1024	2
1024 x 256	4
256 x 1024	4
1024 x 64	16
64 x 1024	16
512 x 512	4

Notes

1. For single-port configuration, this is a single-port configuration.

For single-port configurations, distributed BlockRAM memory has one address port for synchronous writes and always asynchronous reads.

For dual-port configurations, distributed BlockRAM memory has one output for synchronous writes and asynchronous reads, and another port for asynchronous reads. The function generator (FGF) has separate read address inputs (A0, A1, A2, A3, A4) and write address inputs (WE0, WE1, WE2, WE3, WE4, WE5, WE6, WE7, WE8, WE9, WE10, WE11, WE12, WE13, WE14, WE15).

In single-port mode, read and write addresses share the same address bus. In dual-port mode, one function generator (FGF) is connected with shared read and write addresses. The read/write function generator has the 16 inputs (read) connected to the read address inputs and the 16 inputs (write) shared with the 16 read/write port address.

Figure 12, Figure 13, and Figure 14 illustrate various multi-pin configurations.



Figure 12: Multi-Pin Distributed Subcell I/O (Parameter 0)



Figure 13: Multi-Pin Distributed Subcell I/O (Parameter 1)



Figure 14: Single-Pin Distributed Subcell I/O (Parameter 0)

Similar to the FPG configuration, each function generator (LUT) can implement a five input FPG. The configurations are available: *Function*, *Function*, *Function*, *Function*, *Function*, and *Function*. The FPG elements are not available to implement other logic types. FPG elements are located at configuration. Table 10 shows the number of cells occupied by each configuration.

Table 10: FPG Configuration

FPG	Number of cells
Function	1
Function	2
Function	3
Function	4 (up to 4)
Function	5 (up to 5)

Shift Registers

Each function generator can also be configured as a shift register. The write operation is synchronous with a clock input (clk) and an optional enable, as shown in **Figure 46**. A dynamic read access is performed through the read address bus (raddr). The configured read clock and register termination set at least. The read is asynchronous (does not use the storage element or flip-flop) is available to implement a synchronous read. The storage element should always be used with a combinatorial. For example, when building an 8-bit shift register and configuring the datapath to point to the 7th bit, the data bit can be the flip-flop. The stored system performance is improved by using the register clock to set other flip-flops.



Figure 46: Shift Register Configuration

An additional dedicated connection between shift registers allows connecting the output of one shift register to the first input of the next, without using the ordinary I/O output. (See **Figure 47**) Larger shift registers can be built with dynamic access to any bit in the chain. The shift register chaining address (bit0, bit1, and bit2) multiplexers allow all to a 1-bit shift register with addressable access to be implemented (see 4.6.6).

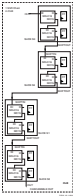


Figure 47: Cascadable Shift Register

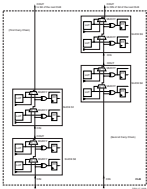


Figure 48: Fast Entry Logic Path

Sum of Products

Each slice of logic has a dedicated OR gate named ORC0. ORC0 aggregates outputs from the slice outputs and the ORC0 from an adjacent slice. The ORC0 gate, with the dedicated Sum of Products (SOP) chain, are designed for implementing logic functions (SOP chains). One input stream ORC0 is not, routed through the four SOP gates to the output of the previous ORC0 in the same slice row. The second stage combines the output of the four SOP chains in the same slice, as shown in Figure 26.

Lookups and MUX00s can implement large AND gates or other combinational logic functions. Figure 26, illustrates Lookups and MUX00 resources configured as a 4-input AND gate.

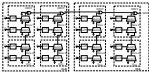


Figure 26: Horizontal Sum-of-Products

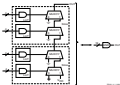


Figure 27: 4-Input AND Gate (48 Inputs)

Figure 1

100%

Each Virtual Disk contains two storage classes. (B)BFS therefore offers multiple features. Each feature cluster has its own file system and its own file system code.

Each of the three above-mentioned methods is based on the fact that the input signals are: through the output matrix, as shown in **Figure 10**, $U(t)$ in neighboring cells can generate also negative or direct currents. The output of the three methods has several advantages: it is easy to implement, it is fast, it is



Abstract

The 16-ounce bottle is important when using 16oz 16oz bottle rather than 16oz 16oz bottle for that bottle is more specific and will be used in the future.

100

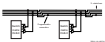
Five horizontal trading scenarios per GDI are provided for example: (a) no income taxes, (b) 10% higher than the average rate, (c) 10% lower than the average rate, (d) 10% higher than the average rate, and (e) 10% lower than the average rate. The scenarios are shown in Figure 28. The information corresponding to the GDI scenario: and multiplier: or GDI income are shown.

Abstract

Table 1 shows the number of leaders/leaders available in each clinical group. The northern leaders leaders is twice the number of L&S members.

Abstract

Iteration	is Greater than 0	Greater than 0
1st iteration	0	0
2nd iteration	0	0
3rd iteration	0	0
4th iteration	0	0
5th iteration	0	0
6th iteration	0	0
7th iteration	0	0
8th iteration	0	0
9th iteration	0	0
10th iteration	0	0
11th iteration	0	0
12th iteration	0	0



© 2004 Blackwell Publishing Ltd *Journal of Internal Medicine* 255: 111–118

CLB/Block Configurations

Table 12 summarizes the logic resources in one CLB. All of the CLBs are identical, and each CLB is also run in implemented form of the configuration tool. **Table 13** shows the available resources in CLBs.

Table 12: Logic Resources in One CLB

Block	CLBs	Flip-Flops	MUXes/ANDs	Arithmetic & Carry Chains	RAM Chains	Distributed RAM/Block	Shift Registers	IOB
1	1	1	1	1	1	10000	10000	1

Table 13: Virtual Vitis Logic Resources Available in All CLBs

Series	CLB Array (Row x Column)	Number of Blocks	Number of LUTs	Max Distributed Multiplexers (4-input Register ports)	Number of Flip-Flops	Number of Carry Chains ⁽¹⁾	Number of RAM Chains ⁽²⁾
integrated	80 x 8	640	640	16,384	640	16	16
integrated	160 x 8	1,280	1,280	32,768	1,280	32	32
integrated	200 x 16	3,200	3,200	81,920	3,200	80	80
integrated	200 x 32	6,400	6,400	163,840	6,400	160	160
ultrascale	200 x 16	16,384	16,384	655,360	16,384	40	40
ultrascale	200 x 32	32,768	32,768	1,310,720	32,768	80	80
ultrascale	200 x 64	65,536	65,536	2,621,440	65,536	160	160
ultrascale	200 x 128	131,072	131,072	5,242,880	131,072	320	320
ultrascale	200 x 256	262,144	262,144	10,485,760	262,144	640	640
ultrascale	200 x 512	524,288	524,288	20,971,520	524,288	1,280	1,280
ultrascale	200 x 1,024	1,048,576	1,048,576	41,943,040	1,048,576	2,560	2,560
ultrascale	200 x 2,048	2,097,152	2,097,152	83,886,080	2,097,152	5,120	5,120

⁽¹⁾ 1000

⁽²⁾ 1. The carry chains and RAM chains can be split or connected.

18-Kbit Block SelectRAM Resources

Introduction

Vitis 2.0 design tools provide a comprehensive 18-Kbit block SelectRAM. These integrated block SelectRAM resources that provide flexible RAM structures implemented in CLBs. Each Vitis 2.0 block SelectRAM is an 18-Kbit non-volatile SelectRAM that has independently decoupled and independently controlled synchronous ports that access a memory storage area. Both ports are functionally identical. CS, Rn, WE, and RDV pins are derived through multiplexers.

Each port has the following general inputs: Clock and Data Enable, Write Enable, Read/Write and Address, as well as separate Read/Write data inputs (for write) and Read/Write data outputs (for read).

Operation as synchronous: the Block SelectRAM behaves like a register. Control, address and data inputs must (and

must) be valid during the setup time window prior to a rising (or falling, as configuration option) clock edge. Data outputs change as a result of the same clock edge.

Configuration

The Vitis 2.0 block SelectRAM supports various configurations, including single- and dual-port RAM and various data widths support values. Supported memory configurations for single- and dual-ports are shown in **Table 14**.

Table 14: Single and Dual-Port Configurations

18K x 1 bit	256 x 18 bits
18K x 2 bit	128 x 18 bits
18K x 4 bit	64 x 18 bits

Single-Port Configuration

In a single-port block, the block's **Read/Write** bus connects to the **Read** memory locations in any of the **SR** or **LR**, **SR** or **LR**, or **SR** or **LR** configurations and to **Read** memory locations in any of the **SR** or **LR**, **SR** or **LR**, or **SR** or **LR** configurations. The advantage of the block, **Read** and **Write** ports is the ability to store a parity bit for each output data. Parity bits must be generated in external circuitry in some logic. In such cases, this width is increased as $1 + N$, $1 + 2$, or $1 + 3$. These extra parity bits are related and behave similarly to the other bits, including the timing parameters. Video applications can use the block's **Read** or **Write** block's **Read/Write** memory to advantage.

Each block's **Read/Write** port is a fully synchronous memory as illustrated in **Figure 28**. Input data, bus, and output data bus widths are identical.



Figure 28 SR-LR Block's Read/Write Memory in Single-Port Mode

Table 28 Single-Port Block Configurations

Port A	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port B	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port A	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port B	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port A	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port B	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port A	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port B	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port A	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port B	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port A	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR
Port B	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR	SR or LR

Block ports are configurable as **SR** or **LR**, **SR** or **LR**, or **SR** or **LR** configurations. The block's **Read** is accessible from Port A or B. If both ports are configured in either **SR** or **LR**, **SR** or **LR**, or **SR** or **LR** configurations, the

Read/Write Configuration

In a read/write block, each port of block's **Read/Write** bus connects to a common **Read/Write** memory resource. These are fully synchronous ports with independent control signals for each port. The data widths of the two ports can be configured independently, providing built-in bus width conversion.

Table 29 illustrates the different configurations available in port A or B.

Each block SelectField cell is a fully synchronous memory as illustrated in [Figure 46](#). The two ports have independent inputs and outputs and are independently tristated.



Figure 46: 16-Bit Block SelectField Cell Block Diagram

Port Signal Ratios

[Table 46](#) shows the inputs and the output signal ratios for the 16-bit block SelectField. Virtual 16-bit SelectField also includes dedicated loading resources to provide an efficient interface with 64-bit, block SelectField, and multiplexers.

Table 46: 16-Bit Block SelectField Port Signal Ratios

Width	Input	Address Bus	Data Bus	Priority Bus
1	16:000	address[0:0]	output[0]	000
2	00000	address[0:1]	output[0:1]	000
3	00000	address[0:2]	output[0:2]	000
4	00000	address[0:3]	output[0:3]	00000
16	00000	address[0:15]	output[0:15]	00000000
64	00000	address[0:63]	output[0:63]	00000000000000

Read/Write Operations

The Virtual 16-bit SelectField read operation is fully synchronous. An address is presented and the read operation is enabled by control signals `rdldn` and `rdldn` in addition to `data` or `data`. Then, depending on data polarity, a rising or falling clock edge causes the stored data to be loaded into output registers.

The write operation is also fully synchronous. Data and address are presented and the write operation is enabled by control signals `wrldn` and `wrldn` in addition to `data` or `data`. Then, again depending on the data input mode, a rising or falling clock edge causes the data to be loaded into the memory cell addresses.

A write operation performs a synchronous read operation. These different options are available, selected by control pins (`en`):

1. `rdldn[0]_rdldn[0]`

The `rdldn[0]_rdldn[0]` option is asynchronous mode. The rising clock edge that writes the data input (`data`) into the memory also enables `data` into the output registers (`data`) as shown in [Figure 46](#).



Figure 46: `rdldn[0]_rdldn[0]` Mode

2. "WRA0_FROST"

The "WRA0_FROST" option is a read-before-write mode.

The same clock edge that writes data input (data_in) into the memory also transfers the prior content of the memory cell addressed into the data output register (data_out), as shown in [Figure 19](#).



Figure 19: WRA0_FROST Mode

3. "W0_Overwrite"

The "W0_Overwrite" option overwrites the content of the output register, regardless of the write operation. The clock edge during this write mode has no effect on the content of the data output register (data_out), rather, the port is configured as "W0_Overwrite", only a read operation loads a new value to the output register (data_out), as shown in [Figure 18](#).



Figure 18: W0_Overwrite Mode

Total Amount of SelectRAM Memory

Table 10 shows the amount of total SelectRAM memory available for each WinbD device. The 18-Kbit SelectRAM blocks are configurable to implement deeper or wider single- or dual-port memory resources.

Table 10 WinbD SelectRAM Memory Availability

Device	Total SelectRAM Memory		
	Blocks	in Words	in Bits
W61000	4	128	230,400
W61000	8	256	460,800
W61000	16	512	921,600
W61000	32	1,024	1,843,200
W61000	64	2,048	3,686,400
W61000	128	4,096	7,372,800
W61000	256	8,192	14,745,600
W61000	512	16,384	29,491,200
W61000	1,024	32,768	58,982,400
W61000	2,048	65,536	117,964,800
W61000	4,096	131,072	235,929,600
W61000	8,192	262,144	471,859,200
W61000	16,384	524,288	943,718,400
W61000	32,768	1,048,576	1,887,436,800

18-Bit x 18-Bit Multiplier

Introduction

A WinbD multiplier block is an 18-bit by 18-bit 2's complement signed multiplier. Using it allows designers to incorporate many embedded multiplier blocks. These multipliers can be used to construct an 18-Kbit-wide SelectRAM resource or can be used independently. They are optimized for high-speed operations and have a lower power consumption compared to an 18-bit x 18-bit multiplier in a logic.

Each SelectRAM memory and multiplier blocks tied to four select RAM lines are shown in **Figure 10**.



Figure 10 (continued)

Figure 10 SelectRAM and Multiplier Blocks

Association With Block SelectRAM Memory

The architecture is designed to allow SelectRAM memory and multiplier blocks tied to the same data bus. Associated interconnect is shown between the SelectRAM and the multiplier. Thus, SelectRAM memory words read only up to 18 bits wide when the multiplier is ready because the multiplier shares inputs with the upper data bits of the SelectRAM memory.

This sharing of the interconnect is optimized for an 18-Kbit-wide SelectRAM resource feeding the multiplier. The use of SelectRAM memory and the multiplier with an associated block address for implementation of a digital signal processor (DSP) multiplier accumulator (MAC) function, which is commonly used in video and video image processing (VLSI) and VLSI digital logic.

Configuration

The multiplier block is an 18-bit by 18-bit signed multiplier (2's complement). Input and output data are signed, and the output is 36 bits. **Figure 11** shows a multiplier block.



Figure 11 (continued)

Figure 11 Multiplier Block

Locations / Organization

Multiplexer organization is identical to the 18K10 VirtexFPGA organization. Every second multiplexer is associated with an 18K10 slice (dataflow resources).

In addition, the built-in multiplexers for the LUT elements have dedicated logic to implement address multiplexers in logic (Point to [Configurable Logic Elements \(LUTs\)](#)).

Table 20: Multiplexer Resource

Device	Columns	Multiplexers	
		Per Column	Total
18K10	2	2	4
18K100	2	4	8
18K1000	8	8	64
18K10000	8	8	64
18K100000	8	16	128
18K1000000	8	16	128
18K10000000	8	16	128
18K100000000	8	16	128
18K1000000000	8	32	256
18K10000000000	8	32	256
18K100000000000	8	32	256

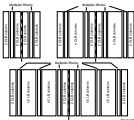


Figure 20: Multiplexers (2 columns, 8 columns, and 8 columns)

Global Clock Multiplexer Buffers

Virtex-5 devices have 16 clock input pins that can also be used as regular user I/Os. Eight clock pins are on the top edge of the device, in the middle of the array, and eight are on the bottom edge, as illustrated in [Figure 37](#).

The global clock multiplexer buffer represents the input to dedicated low-cost clock tree distribution in Virtex-5 devices. Like the clock pins, eight global clock multiplexer buffers are on the top edge of the device and eight are on the bottom edge.



Figure 37: Virtex-5 Clock Pins

Each global clock buffer can either be driven by the clock pins (distributing a clock directly to the device), or driven by the Digital Clock Manager (DCM), discussed in [Digital Clock Manager \(p. 54\)](#), page 54. Each global clock buffer

can also be driven by local interconnects. The DCM has clock outputs that can be connected to global clock buffer inputs, as shown in [Figure 38](#).

Global clock buffers are useful to distribute the clock to some or all synchronous logic elements (such as registers in CLBs, LUTs, and BlockRAM blocks).

Eight global clocks can be used in each quadrant of the Virtex-5 device. Designers should consider the clock distribution based on the device pin configuration and buspin ring (see the Virtex-5 pin listing).



Figure 38: Virtex-5 Clock Distribution Configurations

[Figure 39](#) shows clock distribution in Virtex-5 devices.

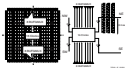


Figure 39: Virtex-5 Clock Distribution

In each quadrant, up to eight clocks are registered in clock rows. A clock row supports up to 16 full row-height up and eight phase. For the output devices, a row clock reset is added, as necessary.

To reduce power consumption, any unused clock function remains static.

Global clocks are offered by dedicated clock buffers (BUFG), which can also be used as general-purpose (BUFGMUX) or to multiplex between two independent clock inputs (BUFGMUX2). The most common configuration option of this element is as a buffer. A BUFG function in this (global) buffer mode, is shown in [Figure 40](#).



Figure 40: Virtual BUFG Function

The (Virtual) global clock buffer (BUFG) can also be configured as clock enable/disable circuit ([Figure 41](#)), as well as a two-input clock multiplexer ([Figure 42](#)). A functional description of these two options is provided below. Each of them can be used in either of two modes, selected by configuration/multiplexers/output falling clock edge.

This section describes the rising clock edge option. For the opposite option, falling clock edge, just change all 'rising' references to 'falling' and all 'High' references to 'Low', except for the description of the `CE` or `EN` inputs. The rising clock edge option uses the `BUFGMUX` architecture principle. The falling clock edge option uses the `BUFGMUX2` architecture principle.

BUFGMUX

If the `CE` input is active (High) prior to the incoming rising clock edge, the Low-to-High-to-Low clock pulse passes through the clock buffer. Any reset change of `CE` during the incoming clock High time has no effect.



Figure 41: Virtual BUFGMUX Function

If the `CE` input is inactive (Low) prior to the incoming rising clock edge, the following clock pulse does not pass through the clock buffer, and the output stays Low. Any reset change of `CE` during the incoming clock High time makes when `CE` must not change during a clock edge window just prior to the rising clock edge on the `BUFGMUX` input. Violating this setup time requirement can result in an undefined reset pulse output.

BUFGMUX2

`BUFGMUX2` can select between two unrelated user inputs, otherwise static. Basically, either an `A` selects the `CE` input or input on `B` selects the `CE` input. Depending on one clock or the other, there is a way that the output High and Low time is easier to determine the `CE` setup time or hold time of other input clock. As long as the presently selected clock is High, any low change will have no effect.



Figure 42: Virtual BUFGMUX2 Function

If the presently selected clock is Low while `A` changes, or if `A` goes Low after `B` has changed, the output is kept Low until the other (pre-selected?) clock has made a transition from High to Low. At that instant, the new clock starts driving the output.

The two clock inputs can be asynchronous with respect to each other and the `S` input can change at any time, except for a short setup time prior to the rising edge of the presently selected clock, that is, prior to the rising edge of the `BUFGMUX2` input `S`. Violating this setup time requirement can result in an undefined multiplexed output.

As `BUFGMUX` devices have 16 global clock multiplexer buffers, [Figure 43](#) shows an overview from `CE` to the `CEMUX`.



Figure 43: Clock Multiplexer Waveform Diagram

- The current clock is `CE`.
- `A` is selected High.
- If `CE` is currently High, the multiplexer selects `A` to `CEMUX` High Low.
- Once `CE` is Low, the multiplexer output stays Low until `CE` transitions High to Low.
- When `CE` transitions from High to Low, the output switches to `B`.
- No glitches or short pulses can appear on the output.

Phase Shifting

The GSC provides additional control over clock skew through either open-drain or line-driven phase shifting. The CLSH, CLASH, CLASHL, and CLASHH outputs, and each phase shifted by 1% of the input clock period relative to each other, allowing coarse phase adjustments.

Fine phase adjustment applies to all GSC output clocks when enabled. The phase shift between the rising edges of CLSH and CLASH is configurable via a significant fraction of the input clock period.

In variable mode, the PHASE_SHIFT value can be dynamically incremented or decremented as determined by PHASEGSC, approximately to PHASE, when the PHASE input is active. [Figure 45](#) illustrates the effect of the phase

shifting. For more information on GSC features, see the Virtual I/Os Guide.

Table 44 lists the phase shifting control pins, when used in variable mode.

Table 44. Fine Phase Shifting Control Pins

Control Pin	Direction	Function
PHASEGSC	in	Incremental adjustment
PHASE	in	Static phase shift
PHASEL	in	Coarse to phase shift
PHASEH	out	Static when completed



Figure 45. Fine Phase Shifting Effects

Two separate components of the phase-shift range must be understood:

- PHASE_SHIFT absolute range
- PHASE_SHIFT_MAXIMAL GSC timing-parameter range

The PHASE_SHIFT absolute is characterized in the following equation:

$$\text{PhaseShift}(ns) = (\text{PHASE_SHIFT_MAXIMAL}) \cdot \text{PHASEGSC}_{\text{MAXIMAL}}$$

The full range of this absolute is always 400 to 400 ns in the practical range series with GSC frequency as constrained by the PHASE_SHIFT_MAXIMAL component, which represents the total delay attributable to the phase shift delay line. Total delay is a function of the number of delay taps used in the circuit. Absolute precision, voltage, and temperature. This absolute range is guaranteed to be as specified under [GSC Pin, I/O Parameters](#).

Absolute range (fixed mode) = $\pm \text{PHASE_SHIFT_MAXIMAL}$

Absolute range (variable mode) = $\pm \text{PHASE_SHIFT_MAXIMAL}$

The reason for the difference between fixed and variable modes is as follows. For variable mode to allow symmetric dynamic ranges from 400ns to 400ns, the GSC uses the "zero phase skew" point as the middle of the delay line, thus shifting the total delay line range in half. In fixed mode, where the PHASE_SHIFT value never changes after configuration, the entire delay line is available for insertion. The other two GSC or CLASH pins (to create either positive or negative skew).

Taking into account these components for consideration, the following series range examples:

- $\text{PHASEGSC}_{\text{MAXIMAL}} = 0 \pm \text{PHASE_SHIFT_MAXIMAL}$ then:
PHASE_SHIFT in fixed mode is limited to ± 400 , and in variable mode is limited to ± 400 .
- $\text{PHASEGSC}_{\text{MAXIMAL}} = \text{PHASE_SHIFT_MAXIMAL}$ then:
PHASE_SHIFT in fixed mode is limited to ± 400 , and in variable mode is limited to ± 400 .
- $\text{PHASEGSC}_{\text{MAXIMAL}} = 0.5 \pm \text{PHASE_SHIFT_MAXIMAL}$ then:
PHASE_SHIFT is limited to ± 400 in other mode.

Operating Modes

The frequency ranges of the GCD input and output clocks depend on the operating mode specified, either low-frequency mode or high-frequency mode, according to

[Table 46, page 161](#) (Powerful values, see [MicroBlaze Software Architecture](#)). The GCDin, GCDenable, GCDout, and GCDclk outputs are not available in high-frequency mode.

Table 46: GCD Frequency Ranges

Output Clock	Low-Frequency Mode		High-Frequency Mode	
	GCDin Input	GCD Output	GCDin Input	GCD Output
BASE, CLKIN0	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100
BASE, CLKIN1	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100	NA	NA
BASE, CLKIN2	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100	NA	NA
BASE	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100
BASE's CLKIN0	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100	BASE_FREQ_0Hz_100	BASEOUT_FREQ_0Hz_100

Locations/Organization

When a GCD is placed on the top and bottom of each base fluid and multiplier column, the number of GCDs depends on the device size, as shown in [Table 46](#).

Table 46: GCD Organization

Device	Columns	GCDs
slvr6000	2	4
slvr6000	2	4
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6
slvr6000	3	6

Active-Interconnect Technology

Local and global Virtual VLSI routing resources are partitioned by speed and timing predictability, as well as by latency (FPGA resource implementation). Virtual VLSI Active-Interconnect Technology is a fully surface programmable routing matrix. All routing resources are segmented to offer the advantages of a hierarchical solution. Virtual VLSI logic resources like CLBs, DFFs, Block RAMs, multipliers, and IOBs are all connected to an identical switch matrix for access to global routing resources, as shown in Figure 46.



Figure 46: Active-Interconnect Technology

Each Virtual VLSI device is represented as an array of switch matrices with logic blocks attached, as illustrated in Figure 47.



Figure 47: Routing Resources

Place-and-route software takes advantage of this regular array to deliver optimum system performance at low configuration times. The segmented routing resources are designed to guarantee 100% routability and functionality while an

incremental design flow that is based on modular logic implementations. Total design time is reduced due to faster and shorter design iterations.

Hierarchical Routing Resources

Most Virtex-5 signals are routed using the global routing resources, which are located in horizontal and vertical routing channels between each switch matrix.

As shown in [Figure 46](#), Virtex-5 has fully-buffered programmable interconnections, with a number of resources located between any two adjacent switch matrix rows or columns. Parallel has minimal impact on the performance of a circuit.

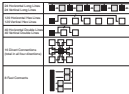


Figure 46-1

Figure 46: Hierarchical Routing Resources

- The long lines are bidirectional wires that distribute signals across the device. Vertical and horizontal long lines span the full height and width of the device.
- The short lines route signals to every first or second switch away in all four directions. Organized in a staggered pattern, the short lines only connect from one end, then the signals can be directed either into endpoints or at the midpoint (provide the nearest source).
- The double lines route signals to every first or second switch away in all four directions. Organized in a staggered pattern, double lines can be driven only at their endpoints. (Double line signals can be connected either at the endpoints or at the midpoint (see below for the reasons)).
- The short connect lines route signals to neighboring sources vertically, horizontally, and diagonally.
- The horizontal lines are the internal Global Interconnections from I/O inputs to I/O outputs.

Distributed Routing

In addition to the global and local routing resources, distributed signals are available.

- There are eight global multi-rate pass transistors (see [Global Clock Multiplexers Buffers](#)).
- Horizontal routing resources are provided to on-chip bus structures. Four partitionable bus lines are provided per CLB row, permitting multiple buses within a row. (See [Global Buffers](#).)
- Two multi-rate on-chip bus resources per slice column (two per CLB column) propagate many-chip MultiCOT output signals vertically to the adjacent slice. (See [Global Buffers Configurations](#).)

- One dedicated NOP chain per slice row (two per CLB row) propagates QPCO output logic signals horizontally to the adjacent slice. (See [Slice Architecture](#).)
- One dedicated shift chain per CLB connects the output of logic in programmable modules throughout the page (with a nonprogrammable mask) vertically through the CLB. (See [Both Regions, page 16](#).)

Creating a Design

Creating Xilinx designs is easy with Xilinx Integrated Synthesis Environment (ISE) development system, which supports advanced design capabilities, including Productive Timing Closure, integrated logic analysis, and the fastest place and route routines in the industry. ISE solutions enable designers to get the performance they need, quickly and easily.

As a result of the ongoing cooperative development efforts between Xilinx and ISE Alliance partners, designers can take advantage of the benefits provided by ISE, namely, place-in-the-programmable logic design process. Xilinx design support options are available in a number of ways to use configurations, inherently proven in the ISE Series.

ISE Alliance

The ISE Alliance solution is designed to play well with an existing design environment. Just using industry standard data formats and routines, these enable, Xilinx products enable Alliance ISE partners to deliver their best design automation capabilities to Xilinx customers, along with the time-to-market benefits of Productive Timing Closure.

ISE Foundation

The ISE Foundation solution delivers the benefits of true FPGAs-based design: true seamlessly integrated design environment. An intuitive project navigator, as well as powerful HDL design and test HDL synthesis tools, ensure that top-quality results are achieved quickly and easily. The ISE Foundation product includes:

- Block Diagram entry using Xilinx (stateful)
- Automatic HDL Testbench generation using Xilinx HDL-Browser
- HDL Simulation using ModelSim SE

Design Flow

Xilinx design flow proceeds as follows:

- Design Entry
- Synthesis
- Implementation
- Verification

Most programmable logic designers focus through these steps, but not more in the process of completing a design.

Design Entry

All Xilinx ISE development systems support the minimum ISE design entry capabilities, ranging from schematic design to automatic HDL design methodologies. Given the high demand of the Xilinx family, designs are created most efficiently using HDLs. To further improve their time to market, many Xilinx customers employ incremental, multi-tier, and incremental-property (IP) design techniques that properly used, these techniques further accelerate through design process.

To enable designers to leverage existing investments in ISE tools, and to ensure high performance design flows, Xilinx jointly develops tools with leading ISE vendors, including:

- Altera®
- Cadence®
- Exemplar®
- Mentor Graphics®
- Model Technology®
- Synopsys®
- Synplify®

Complete information on Alliance Series partners and their associated design flows is available at [www.xilinx.com](#) on the Xilinx Alliance Series webpage.

The ISE Foundation product offers schematic entry and HDL design capabilities as part of an integrated design solution - enabling seamless integrating. These capabilities are powerful, easy to use, and they support the full portfolio of Xilinx programmable logic devices. HDL design capabilities include a color-coded HDL editor with integration, group templates, state diagram entry, and Code generation capabilities.

Synthesize

The ISE Alliance product is engineered to support advanced design flows with the industry's best synthesis tools. Advanced design methodologies include:

- Physical Synthesis
- Incremental synthesis
- HDL Recompiling
- Smart physical mapping

The ISE Foundation product seamlessly integrates synthesis capabilities provided directly from Exemplar, Synopsys, and Synplify. In addition, it includes the capabilities of Xilinx Synthesis Technology.

A benefit of having true seamlessly integrated synthesis engines within an ISE design flow is the ability to apply advanced methods of optimization techniques on designs. Having it ensure that designers can meet with the toughest timing requirements.

Design Implementation

The VHSI SoC design development systems include Virti: the logic-level implementation tools, frequently called "place and route" or "fitting" software. This robust suite of tools enables the creation of an intricate, flexible, tightly integrated design flow that efficiently bridges "logic" and "physical" design elements. This simplifies the task of developing a design, ensuring its automatic, timing requirements, verification (logic- or functional), as well as simplifying the task of analyzing reports generated during the implementation process.

The Virti-Lite implementation process is comprised of Synthesis, translation, mapping, place and route, and configuration for the generation. While the tools handle not individually many design elements to run the entire implementation process with the click of a button. To assist those who prefer to script their design flow, Virti provides them an automated single command line process.

Design Verification

In addition to conventional design verification using static timing analysis or simulation techniques, Virti often provides in-circuit debugging techniques using ChipScope Lite (Integrated Logic Analyzer). The non-invasive nature of Virti FPGAs means that designs can be verified in real time without the need for extensive sets of software simulation cycles.

Furthermore, the system supports post-layout timing information from the design database, and back-annotates this information into the Virti for static verification. The built-in annotation features a variety of powerful Virti techniques, resulting in the industry's most powerful simulation flows. Alternatively, timing-critical portions of a design can be verified using the Virti static timing analyzer or as that party static timing analysis tool like Synopsys Prime TimeTM, by exporting timing data in the EDISM data format.

For in-circuit debugging, ChipScope Lite enables designers to observe the real-time behavior of a device while operating at full system speeds. Logic analysis commands and captured data are transferred between the ChipScope software and Lite mode, within the Virti-Lite FPGA, using industry standard JTAG protocols. These JTAG transactions are driven over an optional download cable (MIL1888 or JTAG), connecting the Virti device to the target system or a PC or workstation.

ChipScope Lite was designed to look and feel like a logic analyzer, making it easy to begin debugging designs in real-time. Modifications to this desired high analysis can be downloaded directly into the system in a matter of minutes.

Other Unique Features of WinVirti Design Flow

Virti design flow features a number of unique capabilities. Among these are efficient incremental FPGAs design flow, a robust capability that is enabled by Virti's exclusive hierarchical floorplanning capabilities. Another powerful design

capability only available in the Virti design flow is "Module Design", part of the Virti suite of tool design tools, which manages extensive design, implementation, and verification of design modules.

Incremental Synthesis

Virti unique hierarchical floorplanning capabilities enable designers to create a project with design flexibility by splitting design elements with the hierarchical "high level", and perform synthesis, verification and implementation processes on the separate high level. By processing the logic in unchanged portions of a design, Virti incremental design makes the high-density design process more efficient.

Virti hierarchical floorplanning capabilities can be specified using the high-level floorplanner or a preferred RTL Synthesizer (over the Virti can also be a list of supported RTL patterns). When used in conjunction with one of the Virti patterns' floorplanners, higher performance results can be achieved, as many synthesis tools use this more performance-driven physical implementation information to establish more aggressive and accurate timing estimates when performing their logic optimizations.

Module Design

Virti innovative module design capabilities take the incremental design process one step further by enabling the designer to delegate responsibility for completing the design synthesis, verification, and implementation to the external "logic level" to an arbitrary number of designers - assigning a specific region within the target FPGA device also use by each of the team members.

This team design capability enables an alternative approach to design modules, enabling the fastest of points the best designer or designer from the module works in simulation to be module works in the Virti-Lite. This unique design methodology also leverages the Virti hierarchical floorplanning capabilities and utilizes the Virti (or Virti primary floorplanner) to manage the efficient implementation of very-high-density FPGAs.

Configuration

Virti's features are configured by loading application specific configuration data into the internal configuration memory. Configuration is verified on unique subset of the device pins, some of which are dedicated, while others can be re-used as general purpose inputs and outputs once verification is complete.

Depending on the system design, external configuration modes are supported, selectable via mode pins. The mode pins 00 (01 and 00) are dedicated pins. An additional pin, HMMAP_0 is used in conjunction with the mode pins to return further user information from just updating configuration. By default, HMMAP_0 is tied high (internal pull-up) when status of the pull-ups on the user I/O pins during reconfiguration. When HMMAP_0 is tied low user I/Os have

perhaps during configuration. Other dedicated pins are $\overline{CS}_{\text{out}}$ (the configuration master pin), $\overline{CS}_{\text{in}}$, $\overline{PROM}_{\text{in}}$, $\overline{B}_{\text{in}}$, and the boundary scan pins $\overline{T0}_{\text{in}}$, $\overline{T0}_{\text{out}}$, $\overline{T0}_{\text{in}}$, and $\overline{T0}_{\text{out}}$. Depending on the configuration mode master, $\overline{CS}_{\text{out}}$ can be an output generated by the FPFA, or an input accepting an externally-generated clock. The configuration pins and boundary scan pins are independent of V_{DD} . The configuration supply ($V_{\text{DD}}^{\text{config}}$) at 1.8V is used for these pins. All configuration pins are $\overline{\text{CMOS}}$. (See [Virtuix VIO Characterization](#).)

A parallel option is available which can be used to force the configuration pins to retain their configuration function even after device configuration is complete. This parallel option is not selected for the configuration pins with the exception of $\overline{CS}_{\text{out}}$, $\overline{PROM}_{\text{in}}$, $\overline{B}_{\text{in}}$, and $\overline{CS}_{\text{in}}$. In operation, this parallel option does not apply to the boundary scan related pins. The parallel feature is valuable in applications which employ partial reconfiguration or reconfiguration on-the-fly.

Configuration Modes

Virtuix supports the following five configuration modes:

- Slave Serial mode
- Master Serial mode
- Slave/SlaveMSP mode
- Master/SlaveMSP mode
- Boundary Scan mode (IEEE 1149.1-1)

A detailed description of configuration modes is provided in the [Virtuix User Guide](#).

Slave Serial Mode

In slave serial mode, the FPFA receives configuration data in 16-bit serial form from a serial PROM or other serial source of configuration data. The $\overline{CS}_{\text{in}}$ pin on the FPFA is an input in this mode. The serial bitstream must be serial write data input pin a short time before each rising edge of the externally-generated $\overline{CS}_{\text{in}}$.

Multiple FPFA's can be serially chained for configuration from a single source. After a particular FPFA has been configured, the data for the next device is routed internally to the $\overline{CS}_{\text{in}}$ pin. The data on the $\overline{CS}_{\text{in}}$ pin changes on the rising edge of $\overline{CS}_{\text{in}}$.

Slave serial mode is selected by applying 0V to the mode pins ($\overline{B0}$, $\overline{B1}$, $\overline{B2}$). It is a pull-up on the mode pins makes slave serial the default mode if the pins are left unconnected.

Master Serial Mode

In master serial mode, the $\overline{CS}_{\text{in}}$ pin is an output pin. It is the Virtuix FPFA device that drives the configuration clock on the $\overline{CS}_{\text{in}}$ pin as a Slave Serial PROM when in turn feeds the serial data to the input. The FPFA generates the data on each rising $\overline{CS}_{\text{in}}$ edge. After the FPFA has been

loaded, the data for the next device is a delay chain is generated on the $\overline{CS}_{\text{in}}$ pin after the rising $\overline{CS}_{\text{in}}$ edge.

The hardware is identical to slave serial except that an internal oscillator is used to generate the configuration clock ($\overline{CS}_{\text{in}}$). A wide range of frequencies can be achieved by $\overline{CS}_{\text{in}}$ using edge delay or a Slave Serial frequency Configuration like that which $\overline{CS}_{\text{in}}$ is a higher frequency for the remainder of the configuration.

Slave/SlaveMSP Mode

This SlaveMSP mode is the fastest configuration option. By using data is written into the Virtix FPFA device with a 16-bit tag controlling the flow of data. An external data source provides a high stream, $\overline{CS}_{\text{in}}$, an write/enable Slave $\overline{CS}_{\text{in}}$ signal and a write signal ($\overline{PROM}_{\text{in}}$). If $\overline{CS}_{\text{in}}$ is present (high) by the FPFA, the data must be read and that 16-bit tag data can also be read using the SlaveMSP mode. If $\overline{PROM}_{\text{in}}$ is asserted, configuration data is read out of the FPFA as part of a readback operation.

After configuration, the pins of the SlaveMSP part can be used as additional user I/O. Alternatively, the part can be returned to parallel operation by asserting the parallel option.

Multiple Virtuix FPFA's can be configured using the SlaveMSP mode, and the mode is set up simultaneously. By connecting multiple devices in this way into the individual $\overline{CS}_{\text{in}}$, data, $\overline{PROM}_{\text{in}}$, and $\overline{CS}_{\text{in}}$ pins at all the devices is parallel. The individual devices are loaded separately by asserting the $\overline{CS}_{\text{in}}$ pin of each device in turn and writing the appropriate data.

Master SlaveMSP Mode

This mode is a master version of the SlaveMSP mode. The device is configured by using a $\overline{CS}_{\text{in}}$ supplied by the Virtix FPFA device. This is similar to the Slave Serial MSP mode except that $\overline{CS}_{\text{in}}$ is supplied by the Virtix FPFA.

Boundary Scan (IEEE 1149.1, IEEE 1149.2) Mode

In boundary scan mode, the device pins are used for serializing the device structure. The configuration is done serially through the $\overline{CS}_{\text{in}}$ pin. The device has $\overline{CS}_{\text{in}}$, $\overline{PROM}_{\text{in}}$, $\overline{B0}$, $\overline{B1}$, $\overline{B2}$, and $\overline{CS}_{\text{in}}$ pins. The device is configured using Boundary Scan compliant with IEEE 1149.1, IEEE standard and the IEEE 1149.2 standard for Boundary-Configurable (BC) devices. The IEEE 1149.2 standard is backward compliant with the IEEE 1149.1-1993 JTAG and also includes. The IEEE Standard 1149.2 for Boundary-Configurable (BC) devices is intended to be programmable, reprogrammable, or tested in the least non physical and logical protocol.

Configuration through the boundary scan port is always available, independent of the mode selection. Returning the boundary scan mode simply turns off the other modes.

Table 35: WinCC Configuration Mode Pin Settings

Configuration Mode ¹⁾	INT	RTN	EN	Global Direction	State Memory	Serial I/O _{max} ²⁾
Master (Serial)	10	10	10	Out	1	Yes
Slave (Serial)	11	1	11	In	1	Yes
Master (Serial/RS485)	10	1	11	Out	0	Yes
Slave (Serial/RS485)	11	1	10	In	0	Yes
Boundary (Flow)	11	10	11	RS485	1	Yes

Notes:

- The INTEN²⁾ pin connects the package housing the INT₁ and INT₂ contacts to the configuration mode; unlike the INTEN²⁾ pin, the INT₁ and INT₂ contacts interface to the package environment.
- Serial I/O refers to the serial data stream. Serial I/O_{max} is used for example, RS485/RS485+ modes; the first device must RTN²⁾ supply along the data stream to the device.

Table 36: Data for the total number of bits required to configure each device.

Table 36: WinCC Bitstream Lengths

Device	Size of Configuration File
64C0000	640,000 bits
64C0001	640,000 bits
64C0002	1,280,000 bits
64C0003	1,280,000 bits
64C0004	1,280,000 bits
64C0005	1,280,000 bits
64C0006	1,280,000 bits
64C0007	1,280,000 bits
64C0008	1,280,000 bits
64C0009	1,280,000 bits
64C0010	1,280,000 bits
64C0011	1,280,000 bits
64C0012	1,280,000 bits
64C0013	1,280,000 bits
64C0014	1,280,000 bits

Configuration Sequence

The configuration of WinCC devices is a three-phase process. First, the configuration memory is erased from the configuration state is loaded into the memory, and finally the logic is effectively starting-up process.

Configuration is automatically information power-up unless it is delayed by the user. The INT₁ pin can be held low using an open-drain driver. An open-drain is required since INT₁ is a bidirectional open-drain pin that is used to carry a WinCC RS485 device while the configuration memory is being loaded. Following the download the pin is transitioned the configuration sequence to each. Thus, configuration is

delivered by generating entry into the phase where state is loaded.

The configuration process commences initially asserting the INTEN₁ signal. The start of the memory clearing phase is signaled by the INT₁ pin going high, and the completion of the write process is signaled by the INTEN₁ pin going high. The Global Direction (GDIR) signal is pulled after the last frame of configuration data is written but before the start-up sequence. The GDIR signal reads all the logic on the device.

The start-up sequence is that once GDIR goes high, the global enable signal (GDIR) is released. This permits device outputs to turn on as device, very low GDIR requires the device (time frame) GDIR signal is released. This permits the internal storage and device to begin changing state in response to the logic and the user state.

The relative timing of these events can be changed via configuration options in software. In addition, the GDIR and GDIR enable events made dependent on the GDIR pin of multiple devices all going high, forcing the devices to start synchronously. The sequence can also be passed at any stage; configuration may occur anytime any or all GDIRs, as well as the GDIR.

Readback

In this mode, configuration data from the WinCC RS485 measurement read back. Readback is supported only in the RS485/RS485+ master and slave and boundary flow mode.

Along with the configuration data, the readback is readback the contents of all registers: distributed RS485/RS485+ and RS485/RS485+ resources. This capability is used for real-time debugging. For more detailed configuration information, see the WinCC User Guide.

Wake and Sleep

Virtuix II devices have an on-chip sleep controller using one or two sets of three logic for tripling data. Sleep/operation/standby (S/O/P) operation. Virtuix software tools offer an optional suspension of the configuration data (downstream with a tripling S/O/P) determined by the designer.

The logic was stored in the PPField by S/O/P instruction and released by a battery connected to the Vcc supply, when the device is not powered. Virtuix II devices can be configured with the corresponding interrupt between, using S/O/P of the configuration modes described previously.

A detailed description of how to use Virtuix on-chip sleep is provided in the Virtuix II User Guide. The iVirtField version also provides specific information on this feature.

Partial Reconfiguration

Partial reconfiguration of Virtuix II devices can be done, provided in other Virtuix iVirtField results or Boundary Scan mode. Instead of reconfiguring the chip and using a full configuration, new data is loaded into a specific area of the chip while the rest of the chip remains in operation. Data is loaded on an on-chip logic, with the on-chip logic controlling a configuration "buffer" of the on-chip (programmable logic, etc.).

Partial reconfiguration is useful for applications that require different designs to be loaded into the same area of chips or that require the ability to change portions of a design without having to reconfigure the entire chip.

Revision History

This section records the change history for this module of the data sheet.

Rev	Version	Revision
1.000.000	1.0	Early version sheet.
1.000.001	1.1	Minor release.
1.000.002	1.2	Added links to the tables for Virtuix II Performance Characteristics and Virtuix II Timing Characteristics sections.
1.000.003	1.3	The description was divided into four modules (per the document style standard). A new was added to Table 1 .
1.000.004	1.4	- Added Logic/Signal Processing Options that target users for general put up and put down modules, was changed to 10 - 100 ns from 100 - 100 ns. - Support unit to open up modules. Reuse the traditional module column format.
1.000.005	1.5	- Added Table 1. - Changed definition of multiple and single integer ranges under Signal/Logic Manager (S/LM). - Added numerous minor edits throughout the module.
1.000.006	1.6	Updated description under Signal/Logic Manager (S/LM), Signal/Logic Manager (S/LM), Signal/Logic Manager (S/LM), and Signal/Logic Manager (S/LM).

Virtuix II Data Sheet

The Virtuix II Data Sheet contains the following modules:

- Table 1. Virtuix II iVirtField [Introduction](#) and [Introduction](#)
 - Table 2. Virtuix II iVirtField [Functional Description](#)
- Table 3. Virtuix II iVirtField [Performance Characteristics](#)
 - Table 4. Virtuix II iVirtField [Timing Characteristics](#)